

5분 대기조 협업 플레이북

ThinQ Workspace Sentinel · W2 뼈대 완성 → W3 본격 개발 · 발행 2026-06-01 · 모듈별 첫 PR 가이드 + 실제 OSS 사례

지금까지 만든 코드는 뼈대. 각 모듈 담당자가 살을 붙이는 게 W3~W6. 본 문서는 "어디서 시작할지 모르겠다"를 없애기 위한 모듈별 첫 PR 예시 + 협업 문화 레퍼런스.

1. 우리가 쓰는 협업 모델 — GitFlow 변형



근거: Vincent Driessen, "A successful Git branching model" (2010). 6주 PoC + 발표 시점(2026-06-25) 명확 → release/main 분리가 적합.

다른 모델 비교

모델	채택 사례	장점	왜 안 맞나
Trunk-based	Google·Facebook·대규모 SaaS	짧은 브랜치·feature flag·빠른 통합	PoC는 통합 빈도 ↓·feature flag 인프라 X
GitHub Flow	GitHub 본사·Heroku	main 하나·단순	release 분리 없어 발표일 고정에 부적합
GitFlow (우리)	Spring·Spotify(초기)·다수 SI	release/hotfix 분리·발표일 명확	적합
GitLab Flow	GitLab·환경 브랜치 다층	staging·production 자연스러움	5명 6주는 과함

2. 4대 OSS 사례에서 배운 패턴

- Kubernetes
OWNERS 파일 + /lgm /approve 봇 명령 2단계 머지
→ 우리: [.github/CODEOWNERS](#)
- Spring Framework
Squash 머지 · PR ↔ Issue/JIRA 1:1 링크
→ 우리: [PR 본문 4섹션 강제 \(어떤 모듈·왜·검증·UIS 경계\)](#)
- React (Meta)
PR 1개 < 400줄 권장 · 큰 refactor도 여러 PR로 쪼개기 (fiber 도입 = 30+ PR)
→ 우리: [본인 모듈에서 하나의 의미 단위만](#)
- Next.js (Vercel)
PR 템플릿 강제 + .changeset 파일 → release notes 자동
→ 우리: [PR 템플릿 4섹션 \(덜 뻑세계\)](#)

추가 레퍼런스

Google · Code Review Developer Guide ([google.github.io/eng-practices](#)) — "LGTM + 1 Nit"
Microsoft · Engineering Fundamentals Playbook — PR 최대 500줄
Atlassian · Comparing Workflows — GitFlow vs GitHub Flow
SmartBear 2008 · 코드 리뷰 효과 측정 — 1시간에 400줄 넘기면 결함 탐지율 급락

3. 모듈별 첫 PR 가이드 (W3 시작점)

Backend — 담당: 이경준

PR-B1 · SSE 공간별 분리

```
git checkout -b feature/backend-sse-by-space-w3
```

현재 단일 ward_a → DB sentinel.spaces 동적 조회. /api/v1/stream/space/{space_id}

PR-B2 · REHVA Calculator 분리

```
git checkout -b feature/backend-rehva-calculator-w3
```

space.py 의 poi().tier() → backend/services/rehva.py 순수 함수로. 단위 테스트 추가.

PR-B3 · Slack/카톡 알림 어댑터 스텝

```
git checkout -b feature/backend-alert-adapter-w3
```

send_slack·send_kakao stub (실제 키 없어도 dry-run)

Frontend — 담당: 김나영

PR-F1 · 병실 상세 모달

```
git checkout -b feature/frontend-room-detail-w3
```

/admin/[roomCode] · 가전 8종 카드 + 센서 6종 + PoI 시계열 (Recharts)

PR-F2 · SSE EventSource 혹화

```
git checkout -b feature/frontend-sse-hook-w3
```

lib/useSentinelStream.ts 커스텀 훅 → /demo · /admin 공유

PR-F3 · 페르소나별 화면 분기

```
git checkout -b feature/frontend-persona-router-w3
```

ICN: 알림 센터 / 원장: 적정성평가 KPI / FM: 가전 운영 8종

Pipeline — 담당: 이우형

PR-P1 · UIS DB read-only 어댑터

```
git checkout -b feature/pipeline-uis-adapter-w3
```

pipeline/adapters/uis_reader.py · SELECT 만 허용. asyncpg 사용.

PR-P2 · 병원체 5종 → 8종 시나리오 추가

```
git checkout -b feature/pipeline-pathogen-scenarios-w3
```

폐렴구균·CDI·옴 시나리오 추가 (seed_scenario)

ML — 담당: 박진영

PR-ML1 · XGBoost 베이스라인 5-Tier 학습

```
git checkout -b feature/ml-xgboost-baseline-w3
```

시물 1만회 → 학습 데이터 생성 → F1·AUPRC·FAR JSON

PR-ML2 · TFT 단기 예측 (옵션 W4)

```
git checkout -b feature/ml-tft-short-w4
```

PyTorch Forecasting · 7일 PoI · attention weight 시각화

DevOps — 담당: 박정빈

PR-D1 · CI hotfix (지금 본 PR)

```
git checkout -b feature/devops-ci-hotfix-w3
```

본 PR로 처리

PR-D2 · VM 모니터링

```
git checkout -b feature/devops-monitoring-w3
```

ntfy/Discord webhook 헬스 알림

PR-D3 · 부하 테스트

```
git checkout -b feature/devops-loadtest-w4
```

k6·locust로 SSE 100 동시 연결 검증

4. PR 만들기 4단계

```
# 1) feature 브랜치
git checkout develop && git pull
git checkout -b feature/<모듈>-<기능>-w3

# 2) 작업 → 커밋
docker compose -f infra/docker-compose.dev.yml up -d
# ...코드 작성...
git add <변경 파일>
git commit -m "feat(<모듈>): <한 줄>"

# 3) push + PR
git push -u origin feature/<모듈>-<기능>-w3
gh pr create --base develop --web

# 4) CI 통과 + 리뷰 1명 → 머지
gh pr checks --watch
```

5. 코드 리뷰 — 무엇을 보나 (Google 기준)

Design	변경이 시스템에 잘 통합되나? 더 단순한 방법은?
Functionality	코드가 실제로 의도한 대로 동작하나?
Complexity	더 단순히 못 만드나? over-engineered 아닌가?
Tests	적절한 자동 테스트가 있나?
Naming	변수·함수·클래스 이름이 명확한가?
Comments	주석이 "왜"를 설명하나 (HOW 아니라)
Style	우리 컨벤션 따랐나 (ruff 통과)

리뷰어 행동 강령

24h 내 1차 응답 (완료 X · 받았다는 사인이라도)
막히면 "이거 어떻게 동작?" 질문 자유롭게
Nit (사소한 것)은 "nit:" prefix → 머지 차단 X
"그냥 다 다시 짜" 류 — 구체적 변경 제안 필수
인신 공격·취향 강요

머지 차단 vs 단순 제안

차단: "Blocker:" / "Must fix:" prefix · 제안: "Nit:" / "Consideration:" prefix

6. 충돌·롤백 절차

충돌 시 rebase

```
git checkout develop && git pull
git checkout feature/<내브랜치>
git rebase develop
# 충돌 해결 후
git add <충돌파일>
git rebase --continue
git push --force-with-lease
```

main 사고 시 즉시 Revert

```
git checkout main && git pull
git revert <bad-commit-sha> -m 1
git push origin main # 자동 배포 → 이전 안정 상태 복귀
```

7. 주차별 마일스톤 (예상 PR 수)

주차	공동 목표	예상 PR
W3 6/2~8	외부신호 → 예측 → 시뮬 통합	B3·F3·P2·ML1·D1 = ~11
W4 6/9~15	Smart Protocol + 시연 영상	F2·F1·B1·ML2 = ~8

W5 6/16~22	다층 게이트·자동 해제·모바일	B2·F1(mobile)·D2 = ~6
W6 6/23~25	최종 발표	hotfix·문서 = ~4

총 ~29 PR, 인당 평균 5~6 PR.

8. 자주 묻는 Q&A:

Q. develop이 자주 바뀌어서 내 PR 충돌남

A. 매일 1회 git fetch + rebase develop 권장. 충돌 작을 때 처리.

Q. CI가 빨간 불인데 뭐가 문제?

A. 본인 로컬 재현: ruff check backend pipeline + PYTHONPATH=. python tests/smoke_scenarios.py. Frontend는 npm install + npm run build.

Q. 본인 모듈 외 디렉토리 수정 필요

A. PR 본문에 이유 명시 + 해당 모듈 담당자 reviewer 지정. 거절되면 별도 PR.

Q. VM에 직접 SSH 들어가서 코드 수정?

A. 절대 금지. 다음 배포에 덮어쓰여 작업 손실.

Q. PR 너무 커졌는데 쪼개도 됨?

A. 적극 권장. React fiber 도입도 30+ PR로 쪼갬.

Q. 발표 임박해서 빨리 머지 필요

A. Hotfix 브랜치 fix/* 로 main 직 PR 가능. Tech Lead 박진영 승인 필수.

5분 대기조 협업 플레이북 v1 · 박진영 (Tech Lead) · GitHub github.com/zln02/thinq-workspace-sentinel (private) · 변경은 docs/team_collab_playbook.md 기준